Name: SAKSHAM SINGHANIA
Roll No.: 23293916033

Cloud Computing
Assignment - 2

Q1. Explain the architecture of a Helm Chart, including its main components.
A Helm Chart is a packaged Kubernetes application. Key components:

- Chart.yaml - metadata: name, version, description
- values.yaml - default config parameters (overridable at deploy time)
- templates/ - Go-templated Kubernetes YAML files (Deployments, Services, etc.)
- charts/ - sub-chart dependencies (e.g. Redis, PostgreSQL)
- crds/ - Custom Resource Definitions to install before templates
- NOTES.txt - post-install instructions printed to terminal

Flow: values.yaml + templates/ → Helm engine renders → pure Kubernetes YAML → applied to cluster.

Q2 Discuss the advantages and limitations of Serverless (FaaS).
Advantages:

- No server management, provider handles infra.
- Auto-scales from zero to thousands of instances.
- Pay-per-execution (no idle cost)
- Fast deployment and event-driven integration.

Limitations:

- Cold start latency (100ms - 2s on first invocation)
- Execution time limits (AWS Lambda max: 15min)
- Stateless by design, states needs external storage.
- Vendor lock-in, hard to migrate
- Difficult to debug / trace across many short-lived invocations.

Q3. Explain how Google Big Query handles large-scale analytics efficiently.

Big Query achieves efficiency through:

- Columnar storage (Capacitor) - reads only queried columns, minimizing I/O.
- Dremel architectures - query tree fans out to thousands of parallel leaf nodes,
- aggregations happen at each level as results travel upward.
- Separation of storage and compute - scales independently, thousands of slots execute in parallel.
- Query optimizer - predicate pushdown, join reordering, broadcast vs shuffle join selection.
- Partioning & clustering - partition pruning skips entire date/range partioning clustering sorts data for faster scans.
- BI Engine - in-memory caching for sub-second repeated queries

__/__/____

Q4. Compare

	Pre-built APIs	AutoML	Custom Models
Skill needed	None	Low	High
Data needed	None	Hundreds-thousands	Millions
Customization	None	Moderate	Full
Time to deploy	Minutes	Hours-days	Weeks-months
Best for	Generic tasks	Domain specific	Novel/research
Example	Vision API, Translate	AutoML tables	Tensorflow/PyTorch

Q5. Explain the complete lifecycle of a serverless function, including trigger, execution and scaling.

1. Deploy - upload code, set runtime, memory, timeout.
2. Register trigger - HTTP, queue event, storage event, cron
3. Trigger fires - invocation event received by platform
4. Cold/warm start - new container initialized (cold) or existing reused (warm)
5. Execute - handler runs with event payload, accesses external services.
6. Return response - result sent back to caller.
7. Auto scaler - concurrent events spawn parallel instances automatically
8. Idle/teardown - container frozen or GC'd after inactivity; billing stops.

Q6. Discuss the role of Helm in DevOps and CI/CD pipelines.

Core roles:

- Templating – single chart deploys to dev/staging/prod via different values.yaml.
- Release Tracking – every deploy is a versioned revision, helm rollback reverts instantly
- Dependency Management – declares all app dependencies in Chart.yaml.

CI/CD pipeline flow:

1. Code push → Docker image built and pushed to registry.
2. helm package → chart pushed to Helm repo
3. helm upgrade --install with environment-specific values file
4. helm test runs in-cluster smoke tests
5. On failure: helm rollback restores previous revision automatically.

GitOps: Tools like Argo CD/Flux watch Git for chart changes and auto-sync the cluster, making Helm charts the deployment artifact and Git the source of truth.

Q7. Explain the working of edge inference with a step-by-step process.

1. Train – full model trained on cloud GPU
2. Optimize – quantization (FP32 → INT8), pruning, knowledge distribution.
3. Convert – export to edge format: TFLite, ONNX, CoreML, TensorRT.
4. Deploy – model pushed to device via OTA update or flashing
5. Capture – sensor collects raw data (camera frame, audio, IMU)
6. Preprocess – resize, normalize, extract features on-device CPU/DSP.
7. Infer – model forward pass runs on NPU/GPU/DSP locally.
8. Post process – raw outputs → labels, bounding boxes, anomaly scores → action.
9. Telemetry – only metadata/results send to cloud, raw data never leaves device.

Key benefit → No network round-trip → sub-millisecond latency, offline capability, data privacy.